

YOLO V8 Model Training

Step 1 : Dataset Loading

```
1 import kagglehub
2
3 # Download latest version
4 path = kagglehub.dataset_download("redzapdos123/indian-driving-dataset-detections-yolov11")
5
6 print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'indian-driving-dataset-detections-yolov11' dataset.
Path to dataset files: /kaggle/input/indian-driving-dataset-detections-yolov11

Step 2 : Dependencies Installation

```
1 %pip install ultralytics torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu118
```

```
Looking in indexes: https://pypi.org/simple, https://download.pytorch.org/whl/cu118
Collecting ultralytics
  Downloading ultralytics-8.3.226-py3-none-any.whl.metadata (37 kB)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.8.0+cu126)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.23.0+cu126)
Requirement already satisfied: torchaudio in /usr/local/lib/python3.12/dist-packages (2.8.0+cu126)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.12.0)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.25.2)
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics_thop-2.0.18-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch) (3.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib==3.3.0->ultral)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib==3.3.0->ultral)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib==3.3.0->ultral)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib==3.3.0->ultral)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib==3.3.0->ultral)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib==3.3.0->ultral)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib==3.3.0->ultral)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultral)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultral)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultral)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultral)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib)
Downloading ultralytics-8.3.226-py3-none-any.whl (1.1 MB)
1.1/1.1 MB 68.6 MB/s eta 0:00:00
Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)
Installing collected packages: ultralytics-thop, ultralytics
```

Step 3 : YAML File Creation/Updation

```

1 import yaml
2 import os
3
4 # Define the path where the dataset was downloaded by kagglehub
5 # Updated path based on the output of cell 3g0m5iqGePzB
6 dataset_base_path = "/kaggle/input/indian-driving-dataset-detections-yolov11"
7 dataset_subdir = "IDDetectionsYOLODataset"
8 dataset_full_path = os.path.join(dataset_base_path, dataset_subdir) # This is the actual path to the dataset content
9
10 # Define the path for the new data.yaml file in a writable location
11 new_data_yaml_path = "/content/data.yaml"
12
13 # Create the content for the new data.yaml file with absolute paths for the base path
14 # and relative paths for train, val, and test
15 data_yaml_content = {
16     'path': dataset_full_path, # Absolute path to the dataset root directory
17     'train': 'train/images', # Relative path to training images from 'path'
18     'val': 'val/images', # Relative path to validation images from 'path'
19     'test': 'test/images', # Relative path to test images from 'path'
20     'nc': 15, # Number of classes in the IDD dataset
21     'names': ['Bus', 'Car', 'Autorickshaw', 'Motorcycle', 'Truck', 'Vehicle', 'Train', 'Parking Sign', 'Traffic Sign',
22 }
23
24 # Write the content to the new data.yaml file
25 with open(new_data_yaml_path, 'w') as f:
26     yaml.dump(data_yaml_content, f)
27
28 print(f"New data.yaml created at: {new_data_yaml_path}")
29
30 # Note: The next cell (e517e00c) should be updated to use new_data_yaml_path for the 'data' argument
31 # and should NOT include explicit 'train', 'val', or 'test' arguments as they are read from the data.yaml

```

New data.yaml created at: /content/data.yaml

```

1 import os
2 import yaml
3
4 # Define the expected dataset path based on the user's input
5 dataset_path = "/kaggle/input/indian-driving-dataset-detections-yolov11/1"
6 data_yaml_path = os.path.join(dataset_path, 'data.yaml')
7
8 # Verify the existence of key directories and data.yaml in the new path
9 expected_dirs = ['images/train', 'images/val', 'labels/train', 'labels/val', 'images/test', 'labels/test'] # Added test
10 all_exist = True
11 for d in expected_dirs:
12     full_path = os.path.join(dataset_path, d)
13     if not os.path.exists(full_path):
14         print(f"Expected directory not found: {full_path}")
15         all_exist = False
16
17 if os.path.exists(data_yaml_path):
18     print(f"data.yaml found at: {data_yaml_path}")
19     # Load and potentially update data.yaml
20     try:
21         with open(data_yaml_path, 'r') as f:
22             data_yaml = yaml.safe_load(f)
23
24         # Update paths in data.yaml to be absolute if they are relative
25         if 'train' in data_yaml and not os.path.isabs(data_yaml['train']):
26             data_yaml['train'] = os.path.join(dataset_path, data_yaml['train'])
27         if 'val' in data_yaml and not os.path.isabs(data_yaml['val']):
28             data_yaml['val'] = os.path.join(dataset_path, data_yaml['val'])
29         # Also update test paths if they exist in data.yaml
30         if 'test' in data_yaml and not os.path.isabs(data_yaml['test']):
31             data_yaml['test'] = os.path.join(dataset_path, data_yaml['test'])
32
33         # Save the updated data.yaml
34         with open(data_yaml_path, 'w') as f:
35             yaml.dump(data_yaml, f)
36         print("data.yaml updated with absolute paths.")
37     except Exception as e:
38         print(f"Error processing data.yaml: {e}")
39         all_exist = False
40
41 else:
42     print(f"data.yaml not found at: {data_yaml_path}")
43     all_exist = False

```

```

45
46 if all_exist:
47     print("Dataset structure appears correct.")
48 else:
49     print("Dataset structure is incomplete or incorrect.")
50
51 # Note: We are no longer checking Google Drive in this cell
52 # The 'all_exist' variable now reflects the status of the new path

```

```

Expected directory not found: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versior
Expected directory not found: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versior
Expected directory not found: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versior
Expected directory not found: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versior
Expected directory not found: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versior
Expected directory not found: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versior
data.yaml not found at: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/ID
Dataset structure is incomplete or incorrect.

```

▼ Step 4 : Dataset / Directory Inspection

```

1 import os
2
3 # Define the path where the dataset was downloaded by kagglehub
4 # Updated path based on the output of cell 3g0m5iqGePzB
5 dataset_base_path = "//kaggle/input/indian-driving-dataset-detections-yolov11"
6
7 print(f"Inspecting directory structure at: {dataset_base_path}")
8
9 # List contents of the base directory
10 print(f"\nContents of {dataset_base_path}:")
11 for item in os.listdir(dataset_base_path):
12     print(item)
13
14 # If there's a subdirectory like 'IDDDetectionsYOLODataset', inspect its contents as well
15 dataset_subdir = os.path.join(dataset_base_path, "IDDDetectionsYOLODataset")
16 if os.path.exists(dataset_subdir):
17     print(f"\nContents of {dataset_subdir}:")
18     for item in os.listdir(dataset_subdir):
19         print(item)
20     # Further inspect the expected images and labels directories
21     expected_subdirs = ['train', 'val', 'test']
22     for subdir in expected_subdirs:
23         images_path = os.path.join(dataset_subdir, subdir, 'images')
24         labels_path = os.path.join(dataset_subdir, subdir, 'labels')
25         if os.path.exists(images_path):
26             print(f"\nContents of {images_path} (first 5 items):")
27             try:
28                 for i, item in enumerate(os.listdir(images_path)):
29                     if i < 5:
30                         print(item)
31                     else:
32                         break
33             except Exception as e:
34                 print(f"Could not list contents: {e}")
35         else:
36             print(f"\n{images_path} not found.")
37
38         if os.path.exists(labels_path):
39             print(f"\nContents of {labels_path} (first 5 items):")
40             try:
41                 for i, item in enumerate(os.listdir(labels_path)):
42                     if i < 5:
43                         print(item)
44                     else:
45                         break
46             except Exception as e:
47                 print(f"Could not list contents: {e}")
48         else:
49             print(f"\n{labels_path} not found.")

```

```
Inspecting directory structure at: //kaggle/input/indian-driving-dataset-detections-yolov11
```

```
Contents of //kaggle/input/indian-driving-dataset-detections-yolov11:
IDDDetectionsYOLODataset
```

```
Contents of //kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset:
license.md
ReadMe.md
data.yaml
val
test

```

```
train
```

```
Contents of //kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/train/images (first 5 items):
```

```
BLR-2018-05-17_15-56-30_sideLeft_000762_r.jpg
BLR-2018-05-29_11-08-34_sideLeft_000213_r.jpg
BLR-2018-05-29_10-59-01_sideRight_000780_r.jpg
BLR-2018-06-20-07-01-47_part_14_0000431.jpg
BLR-2018-05-17_16-32-30_frontNear_0000600.jpg
```

```
Contents of //kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/train/labels (first 5 items):
```

```
BLR-2018-05-11_13-42-30_sideRight_right_0000583.txt
BLR-2018-05-22_11-54-07_sideLeft_000483_r.txt
BLR-2018-05-22_12-04-54_sideLeft_0010185.txt
HYD-2018-06-11_13-54-41_frontNear_left_0001785.txt
BLR-2018-06-20-07-01-47_part_30_0000778.txt
```

```
Contents of //kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/val/images (first 5 items):
```

```
BLR-2018-06-05_08-39-44_sideRight_0003480.jpg
BLR-2018-04-19_17-16-55_frontFar_0000660.jpg
HYD-2018-06-11_13-44-41_0006892.jpg
BLR-2018-05-03_14-05-29_rearNear_right_0005502.jpg
BLR-2018-06-05_09-06-38_2018-06-05_9-16-30-221711_leftImg8bit.jpg
```

```
Contents of //kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/val/labels (first 5 items):
```

```
BLR-2018-06-05_08-39-44_frontNear_000834_r.txt
BLR-2018-06-20-05-53-14_part_5_0001281.txt
BLR-2018-05-03_14-15-50_sideRight_right_0004200.txt
BLR-2018-05-17_15-23-02_sideRight_0006360.txt
BLR-2018-06-29-10-07-38_part_2_0001590.txt
```

```
Contents of //kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/test/images (first 5 items):
```

```
BLR-2018-05-11_13-53-01_0003450.jpg
BLR-2018-05-03_14-05-29_sideLeft_left_0002011.jpg
BLR-2018-05-07_12-36-24_rearNear_0000480.jpg
BLR-2018-05-17_15-23-02_frontNear_0002700.jpg
BLR-2018-04-25_13-30-42_frontNear_left_0008010.jpg
```

```
Contents of //kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/test/labels (first 5 items):
```

```
BLR-2018-06-22-05-05-26_part_26_0001500.txt
HYD-2018-06-11_13-54-41_frontNear_left_0003885.txt
BLR-2018-05-23_12-56-25_sideRight_0006360.txt
BLR-2018-05-03_14-26-52_sideLeft_left_0007257.txt
BLR-2018-05-03_14-05-29_sideRight_right_0008874.txt
```

Step 5 : Model Configuration & Training

No. of Epochs : 5 Epoch

```
1 from ultralytics import YOLO
2 import os
3
4 # Define the path to the data.yaml file based on the dataset verification
5 # We will use the path where data.yaml was found and updated
6 data_yaml_path = '/content/data.yaml'
7
8 # Define the correct absolute path to the dataset directory
9 dataset_base_path = "/root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/"
10
11
12 # Initialize the YOLO model (using yolov8m for this task)
13 model = YOLO('yolov8m.pt')
14
15 # Define training parameters
16 # Using parameters from the user's reference command, adjusting for yolov8m and T4 GPU
17 train_args = {
18     'data': data_yaml_path, # Keep data.yaml path, which contains the dataset paths
19     'model': 'yolov8m.pt', # Specify yolov8m model
20     'epochs': 5,
21     'batch': 16, # Batch size adjusted for T4 GPU memory
22     'imgsz': 640,
23     'lr0': 0.005,
24     'weight_decay': 0.0005,
25     'momentum': 0.937,
26     'mosaic': 1.0,
27     'mixup': 0.1,
28     'hsv_v': 0.4,
29     'hsv_s': 0.7,
30     'box': 0.05,
31     'cls': 0.5,
32     'dfl': 1.0,
33     'optimizer': 'AdamW', # Using AdamW optimizer
34     'cos_lr': True, # Cosine learning rate scheduler
```

```
35     'close_mosaic': 10, # Disable mosaic for the last 10 epochs
36     # Removed explicit 'train', 'val', and 'test' paths as they are not valid arguments for model.train()
37     # These paths are read from the data.yaml file specified by the 'data' argument
38 }
39
40 # Start training
41 results = model.train(**train_args)
42
43 print("Training complete.")
44
45 # Removed model export code from this cell
```



```

Ultralytics 8.3.226 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)
engine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=randaugument, batch=16, bgr=0.0, box=0.05, cache=Fa
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100% 755.1KB 20.
Overriding model.yaml nc=80 with nc=15

      from  n  params module  arguments
0         -1 1    1392 ultralytics.nn.modules.conv.Conv [3, 48, 3, 2]
1         -1 1    41664 ultralytics.nn.modules.conv.Conv [48, 96, 3, 2]
2         -1 2    111360 ultralytics.nn.modules.block.C2f [96, 96, 2, True]
3         -1 1    166272 ultralytics.nn.modules.conv.Conv [96, 192, 3, 2]
4         -1 4    813312 ultralytics.nn.modules.block.C2f [192, 192, 4, True]
5         -1 1    664320 ultralytics.nn.modules.conv.Conv [192, 384, 3, 2]
6         -1 4    3248640 ultralytics.nn.modules.block.C2f [384, 384, 4, True]
7         -1 1    1991808 ultralytics.nn.modules.conv.Conv [384, 576, 3, 2]
8         -1 2    3985920 ultralytics.nn.modules.block.C2f [576, 576, 2, True]
9         -1 1    831168 ultralytics.nn.modules.block.SPPF [576, 576, 5]
10        -1 1         0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
11      [-1, 6] 1         0 ultralytics.nn.modules.conv.Concat [1]
12        -1 2    1993728 ultralytics.nn.modules.block.C2f [960, 384, 2]
13        -1 1         0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
14      [-1, 4] 1         0 ultralytics.nn.modules.conv.Concat [1]
15        -1 2    517632 ultralytics.nn.modules.block.C2f [576, 192, 2]
16        -1 1    332160 ultralytics.nn.modules.conv.Conv [192, 192, 3, 2]
17      [-1, 12] 1         0 ultralytics.nn.modules.conv.Concat [1]
18        -1 2    1846272 ultralytics.nn.modules.block.C2f [576, 384, 2]
19        -1 1    1327872 ultralytics.nn.modules.conv.Conv [384, 384, 3, 2]
20      [-1, 9] 1         0 ultralytics.nn.modules.conv.Concat [1]
21        -1 2    4207104 ultralytics.nn.modules.block.C2f [960, 576, 2]
22     [15, 18, 21] 1    3784381 ultralytics.nn.modules.head.Detect [15, [192, 384, 576]]

Model summary: 169 layers, 25,865,005 parameters, 25,864,989 gradients, 79.1 GFLOPs

Transferred 469/475 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt to 'yolo11n.pt': 100% 5.4
AMP: checks passed
train: Fast image access (ping: 0.0±0.0 ms, read: 713.7±1507.3 MB/s, size: 569.6 KB)
train: Scanning /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/IDDDetecti
train: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/IDDDetectionsYOLODa
train: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/IDDDetectionsYOLODa
train: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/IDDDetectionsYOLODa
train: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/IDDDetectionsYOLODa
train: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/IDDDetectionsYOLODa
train: New cache created: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/ID
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_aver
val: Fast image access (ping: 0.0±0.0 ms, read: 1375.3±1067.1 MB/s, size: 588.1 KB)
val: Scanning /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/IDDDetectio
val: New cache created: /root/.cache/kagglehub/datasets/redzapdos123/indian-driving-dataset-detections-yolov11/versions/1/ID
Plotting labels to /content/runs/detect/train6/labels.jpg...
optimizer: AdamW(lr=0.005, momentum=0.937) with parameter groups 77 weight(decay=0.0), 84 weight(decay=0.0005), 83 bias(deca
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /content/runs/detect/train6
Starting training for 5 epochs...

  Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
  1/5      8.23G    0.01068    1.487    0.9057      377      640: 100% 2098/2098 1.7it/s 20:38
      Class  Images  Instances  Box(P          R      mAP50  mAP50-95): 100% 132/132 1.9it/s
      all      4196    49838    0.477    0.255    0.239    0.138

  Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
  2/5      7.32G    0.01029    1.317    0.8895      449      640: 100% 2098/2098 1.7it/s 20:17
      Class  Images  Instances  Box(P          R      mAP50  mAP50-95): 100% 132/132 1.9it/s
      all      4196    49838    0.686    0.258    0.315    0.197

  Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
  3/5      7.39G    0.009606    1.164    0.8473      506      640: 100% 2098/2098 1.7it/s 20:19
      Class  Images  Instances  Box(P          R      mAP50  mAP50-95): 100% 132/132 1.9it/s
      all      4196    49838    0.661    0.295    0.349    0.216

  Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
  4/5      7.28G    0.008994    1.031    0.8098      354      640: 100% 2098/2098 1.7it/s 20:34
      Class  Images  Instances  Box(P          R      mAP50  mAP50-95): 100% 132/132 1.9it/s
      all      4196    49838    0.592    0.351    0.382    0.241

  Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
  5/5      7.73G    0.008449    0.9139    0.7797      330      640: 100% 2098/2098 1.7it/s 20:23
      Class  Images  Instances  Box(P          R      mAP50  mAP50-95): 100% 132/132 1.9it/s
      all      4196    49838    0.69     0.4     0.431    0.283

5 epochs completed in 1.810 hours.
Optimizer stripped from /content/runs/detect/train6/weights/last.pt, 52.0MB
Optimizer stripped from /content/runs/detect/train6/weights/best.pt, 52.0MB

```

```

1 from ultralytics import YOLO
2 import os
3
4 # Load the best trained model from the previous cell's run
5 # The 'best.pt' model is usually saved in the 'runs/detect/trainX/weights' directory
6 # where X is the run number. You might need to adjust the path based on your output.

```

```

7 # A more robust way is to get the path from the 'results' object if available,
8 # but assuming the best model is saved in a predictable location:
9 # For example, if your training run is 'train6', the best model is at:
10 # '/content/runs/detect/train6/weights/best.pt'
11
12 # Replace 'trainX' with the actual run directory name from your training output
13 # You can find this in the output of the training cell, e.g., 'Logging results to runs/detect/train6'
14 trained_model_path = '/content/runs/detect/train6/weights/best.pt' # Update 'train6' as needed
15
16 try:
17     # Load the trained model
18     model = YOLO(trained_model_path)
19
20     # Export the trained model to ONNX, PT, and SavedModel (.pb) formats
21     print(f"Exporting model from: {trained_model_path}...")
22
23     onnx_path = model.export(format='onnx', name='yolov8m_epoch5')
24     print(f"ONNX model exported to: {onnx_path}")
25
26     # Removed export to 'pt' format as the model is already in this format
27     # pt_path = model.export(format='pt', name='yolov8m_epoch5')
28     # print(f"PT model exported to: {pt_path}")
29
30     saved_model_path = model.export(format='saved_model', name='yolov8m_epoch5_saved_model')
31     print(f"SavedModel (.pb) exported to: {saved_model_path}")
32
33     print("Model exported to ONNX and SavedModel (.pb) formats.")
34
35 except FileNotFoundError:
36     print(f"Error: Trained model not found at {trained_model_path}.")
37     print("Please ensure the training in the previous cell completed successfully and the 'trained_model_path' variable is correct.")
38 except Exception as e:
39     print(f"An error occurred during model export: {e}")

```

Exporting model from: /content/runs/detect/train6/weights/best.pt...
WARNING ⚠️ **ONNX**: Predict a YouTube video using a pretrained segmentation model at image size 320:
 Model summary (fused): 92 layers, 25,848,445 parameters, 0 gradients, 78.7 GFLOPs
 Arguments received: ['yolo', '-f', '/root/.local/share/jupyter/runtime/kernel-d26547ae-0ced-433c-8352-b97f45a1be3189d7d7c1031003']
 Starting from /content/runs/detect/train6/weights/best.pt with input shape (1, 3, 640, 640) BCHW and output shape (1, 3, 320, 320)
requirements: Ultralytics requirement ['onnxruntime'] not found, attempting AutoUpdate...
 Using Python 3.12.12 environment at: /usr
 Resolved 9 packages in 136ms
 Prepared 9 packages (onnxruntime) is one of ['detect', 'pose', 'segment', 'obb', 'classify']
 Installed 1 package (onnxruntime) is one of ['export', 'val', 'track', 'benchmark', 'train', 'predict']
 + onnxruntime==8.2.0 (python 3.12.12) any number of custom 'arg=value' pairs like 'imgsz=320' that override defaults.
 See all ARGS at <https://docs.ultralytics.com/usage/cfg> or with 'yolo cfg'
requirements: AutoUpdate success ✓ 0.6s
WARNING ⚠️ **ONNX**: Train detection model for 10 epochs with a minimal learning rate of 0.01
 yolo train data=coco8.yaml model=yolo11n.pt epochs=10 lr0=0.01
ONNX: Predict a YouTube video using a pretrained segmentation model at image size 320:
 WARNING ⚠️ **ONNX**: Predict a YouTube video using a pretrained segmentation model at image size 320:
 Model summary (fused): 92 layers, 25,848,445 parameters, 0 gradients, 78.7 GFLOPs
ONNX: export success ✓ 4.1s, saved as '/content/runs/detect/train6/weights/best.onnx' (98.8 MB)
 3. Val a pretrained detection model at batch-size 1 and image size 640:
 yolo val model=yolo11n.pt data=coco8.yaml batch=1 imgsz=640
 Export complete (7.0s)
 Results saved to /content/runs/detect/train6/weights
 Predict a YouTube video using a pretrained segmentation model at image size 320:
 Validate YOLO export model=yolo11n.pt data=coco8.yaml batch=1 imgsz=640 data=/content/data.yaml
 Visualize: <https://netron.app>
ONNX: Ultralytics detected 7.0.0 at /content/runs/detect/train6/weights/best.onnx
 Ultralytics 7.0.0 is the latest version. To update to the latest version, run 'pip install --upgrade ultralytics' (Internet CPU speed 200MHz)
 Model summary (fused): 92 layers, 25,848,445 parameters, 0 gradients, 78.7 GFLOPs
PyTorch: Starting from /content/runs/detect/train6/weights/best.pt with input shape (1, 3, 640, 640) BCHW and output shape (1, 3, 320, 320)
 yolo help
TensorFlow SavedModel: starting export with tensorflow 2.19.0...
 yolo version
ONNX: Starting export with onnx 1.20.0rc1 opset 22...
 WARNING ⚠️ **ONNX**: Simplifier failure: module 'onnx.helper' has no attribute 'float32_to_bfloat16'
ONNX: export success ✓ 5.9s, saved as '/content/runs/detect/train6/weights/best.onnx' (98.8 MB)
ERROR ✖️ **TensorFlow SavedModel**: export failure 13.7s: module 'onnx.helper' has no attribute 'float32_to_bfloat16'
 An error occurred during model export: module 'onnx.helper' has no attribute 'float32_to_bfloat16'
 Docs: <https://docs.ultralytics.com>
 Solutions: <https://docs.ultralytics.com/solutions/>
 Community: <https://community.ultralytics.com>

✓ No. of Epochs 10 Epoch

```

1 from ultralytics import YOLO
2 import os
3
4 # Define the path to the data.yaml file based on the dataset verification
5 # We will use the path where data.yaml was found and updated
6 data_yaml_path = '/content/data.yaml'
7
8
9 # Initializing the YOLO model (using yolov8m for this task)

```



```
9 # initialize the YOLO model (using yolov8m for this task)
10 model = YOLO('yolov8m.pt')
11
12 # Define training parameters
13 # Using parameters from the user's reference command, adjusting for yolov8m and T4 GPU
14 train_args = {
15     'data': data_yaml_path,
16     'model': 'yolov8m.pt', # Specify yolov8m model
17     'epochs': 10,
18     'batch': 16, # Batch size adjusted for T4 GPU memory
19     'imgsz': 640,
20     'lr0': 0.005,
21     'weight_decay': 0.0005,
22     'momentum': 0.937,
23     'mosaic': 1.0,
24     'mixup': 0.1,
25     'hsv_v': 0.4,
26     'hsv_s': 0.7,
27     'box': 0.05,
28     'cls': 0.5,
29     'dfl': 1.0,
30     'optimizer': 'AdamW', # Using AdamW optimizer
31     'cos_lr': True, # Cosine learning rate scheduler
32     'close_mosaic': 10 # Disable mosaic for the last 10 epochs
33 }
34
35 # Start training
36 results = model.train(**train_args)
37
38 print("Training complete.")
```


Ultralytics 8.3.226 Python-3.12.12 torch-2.8.0+cu126 CUDA:0 (Tesla T4, 15095MiB)
engine/trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=randaugument, batch=16, bgr=0.0, box=0.05, cache=False
 Downloading <https://ultralytics.com/assets/Arial.ttf> to '/root/.config/Ultralytics/Arial.ttf': 100% ————— 755.1KB 132
 Overriding model.yaml nc=80 with nc=15

		from	n	params	module	arguments
0		-1	1	1392	ultralytics.nn.modules.conv.Conv	[3, 48, 3, 2]
1		-1	1	41664	ultralytics.nn.modules.conv.Conv	[48, 96, 3, 2]
2		-1	2	111360	ultralytics.nn.modules.block.C2f	[96, 96, 2, True]
3		-1	1	166272	ultralytics.nn.modules.conv.Conv	[96, 192, 3, 2]
4		-1	4	813312	ultralytics.nn.modules.block.C2f	[192, 192, 4, True]
5		-1	1	664320	ultralytics.nn.modules.conv.Conv	[192, 384, 3, 2]
6		-1	4	3248640	ultralytics.nn.modules.block.C2f	[384, 384, 4, True]
7		-1	1	1991808	ultralytics.nn.modules.conv.Conv	[384, 576, 3, 2]
8		-1	2	3985920	ultralytics.nn.modules.block.C2f	[576, 576, 2, True]
9		-1	1	831168	ultralytics.nn.modules.block.SPPF	[576, 576, 5]
10		-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11		[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
12		-1	2	1993728	ultralytics.nn.modules.block.C2f	[960, 384, 2]
13		-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14		[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
15		-1	2	517632	ultralytics.nn.modules.block.C2f	[576, 192, 2]
16		-1	1	332160	ultralytics.nn.modules.conv.Conv	[192, 192, 3, 2]
17		[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
18		-1	2	1846272	ultralytics.nn.modules.block.C2f	[576, 384, 2]
19		-1	1	1327872	ultralytics.nn.modules.conv.Conv	[384, 384, 3, 2]
20		[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
21		-1	2	4207104	ultralytics.nn.modules.block.C2f	[960, 576, 2]
22		[15, 18, 21]	1	3784381	ultralytics.nn.modules.head.Detect	[15, [192, 384, 576]]

Model summary: 169 layers, 25,865,005 parameters, 25,864,989 gradients, 79.1 GFLOPs

Transferred 469/475 items from pretrained weights

Freezing layer 'model.22.dfl.conv.weight'

AMP: running Automatic Mixed Precision (AMP) checks...

Downloading <https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt> to 'yolo11n.pt': 100% ————— 5.4

AMP: checks passed

train: Fast image access (ping: 0.7±0.1 ms, read: 36.5±5.3 MB/s, size: 569.6 KB)

train: Scanning /kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/train/labels... 33569 image

train: /kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/train/images/15-07-18-upload_0017293

train: /kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/train/images/BLR-2018-06-22-05-05-26

train: /kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/train/images/HYD-2018-06-15_11-55-11

train: /kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/train/images/HYD-2018-08-24_13-02-50

train: /kaggle/input/indian-driving-dataset-detections-yolov11/IDDDetectionsYOLODataset/train/images/HYD-2018-08-24_13-22-56

```

1 from ultralytics import YOLO
2 import os
3
4 # Load the best trained model from the previous cell's run
5 # The 'best.pt' model is usually saved in the 'runs/detect/trainX/weights' directory
6 # where X is the run number. You might need to adjust the path based on your output.
7 # A more robust way is to get the path from the 'results' object if available,
8 # but assuming the best model is saved in a predictable location:
9 # For example, if your training run is 'train6', the best model is at:
10 # '/content/runs/detect/train6/weights/best.pt'
11
12 # Replace 'trainX' with the actual run directory name from your training output
13 # You can find this in the output of the training cell, e.g., 'Logging results to runs/detect/train6'
14 # Note: The run directory name might change with each training run.
15 # You can find the exact path in the output of the training cell.
16 trained_model_path = '/content/runs/detect/train2/weights/best.pt' # Update 'train2' as needed
17
18 try:
19     # Load the trained model
20     model = YOLO(trained_model_path)
21
22     # Export the trained model to ONNX and PT formats after 10 epochs
23     print(f"Exporting model from: {trained_model_path}...")
24
25     onnx_path = model.export(format='onnx')
26     print(f"ONNX model exported to: {onnx_path}")
27
28     # Remove this line as the model is already in PT format
29     # pt_path = model.export(format='pt')
30     # print(f"PT model exported to: {pt_path}")
31
32     print("Model exported to ONNX format after 10 epochs.")
33
34 except FileNotFoundError:
35     print(f"Error: Trained model not found at {trained_model_path}.")
36     print("Please ensure the training in the previous cell completed successfully and the 'trained_model_path' variable")
37 except Exception as e:
38     print(f"An error occurred during model export: {e}")

```

Exporting model from: /content/runs/detect/train2/weights/best.pt...
 Ultralytics 8.3.226 Python-3.12.12 torch-2.8.0+cu126 GPU (Intel Xeon CPU @ 2.00GHz)
 Model summary (fused): 169 layers, 25,865,005 parameters, 0 gradients, 78.7 GFLOPs
 Class Images Instances Box(P R mAP50 mAP50-95): 100% 132/132 1.8it/s

✓ No. of Epochs : 15 Epoch

10 epochs completed in 3.548 hours.